

NAVEEN AUTOMATION LABS

DRIVEN BY CODE, DEFINED BY QUALITY



PLAYWRIGHT FRAMES

iFrame & Frame Handling

Date: _____

What are Frames?

- Frames (iframe) are **separate HTML documents** embedded in a page
- Regular locators **cannot access** elements inside frames!
- Need **special methods** to interact with frame elements

KEY CONCEPT!

Think of iframes as "page within a page" - they have their own DOM!

You must switch to the frame context before interacting with elements inside it.

Three Ways to Handle Frames

1. `frameLocator()` **RECOMMENDED**

Locator-based, auto-waits, safest approach

2. `frame()` ⚠ Not Recommended

Returns Frame object, doesn't wait, can return null

3. `frames()` 📊 For Counting

Get all frames on the page

★ Method 1: `frameLocator()` - BEST!

- **Auto-waits** for frame to load
- **Locator-based** - works with any selector
- **Recommended** approach for frame handling!

{ } `frameLocator()` Examples (TypeScript)

```
import { Browser, chromium, Page } from '@playwright/test';
(async () => {
  let browser: Browser = await chromium.launch({
    headless: false,
    channel: 'chrome'
  });
  let page: Page = await browser.newPage();
  // Navigate to page with iframe
  await page.goto('https://www.formsite.com/templates/');
  // Get frameLocator - auto-waits for frame!
  let frameEle = page.frameLocator('iframe[id*="frame-one"]');
  // Now interact with elements inside the frame
  await frameEle.locator('#RESULT_TextField-1').fill('Naveen Automation');
  // Access main page element (outside frame)
  let vehicleHeader = await page
    .locator('h3.details__form-preview-title')
    .innerText();
  console.log(vehicleHeader);
  await browser.close();
})();
```

💡 FRAMELOCATOR() TIPS:

Can use any selector:

✓ `page.frameLocator('#frame-id')` - By ID

✓ `page.frameLocator('iframe[name="main"]')` - By name

✓ `page.frameLocator('iframe[src*="form"]')` - By src

✓ `page.frameLocator('iframe.frame-class')` - By class

Auto-waits! No need for manual waits! ✨

⚠️ Method 2: `frame()` - Not Recommended

- Doesn't wait for frame to load
- Can return `null` if frame not found
- Only works with `name or url` attributes

{ } `frame()` Example - Use with Caution!

```
import { Browser, chromium, Page, Frame } from '@playwright/test';
(async () => {
  let browser: Browser = await chromium.launch({
    headless: false,
    channel: 'chrome'
  });
  let page: Page = await browser.newPage();
  await page.goto('https://www.londonfreelance.org/courses/frames/');
  // Get frame by name - can return null!
  let topFrame: Frame | null = page.frame({ name: 'main' });
  // ⚠️ Must check for null!
  if (!topFrame) {
    throw new Error('Frame not found');
  }
})
```

```
// Use optional chaining (?) for safety
let header = await topFrame?.locator('h2').innerText();
console.log(header);

// Get frame by URL
let frameByUrl = page.frame({ url: /. *courses.*/ });
await browser.close();
})();
```

💡 WHY FRAMELOCATOR() IS BETTER:

frameLocator():

- ✓ Auto-waits for frame
- ✓ Works with any selector
- ✓ Never returns null
- ✓ Safer and more reliable

frame():

- ✗ Doesn't wait
- ✗ Only name/url selectors
- ✗ Can return null
- ✗ Needs manual null checks

📊 Method 3: frames() - Get All Frames

- Returns array of all Frame objects
- Useful to count frames or loop through them
- Can get frame names and URLs

`{ }` frames() - Get All Frames

```
import { Browser, chromium, Page, Frame } from '@playwright/test';
(async () => {
  let browser: Browser = await chromium.launch({
    headless: false,
```

```

channel: 'chrome'
});
let page: Page = await browser.newPage();
await page.goto('https://www.londonfreelance.org/courses/frames/');
// Get all frames on the page
let allFrames: Frame[] = page.frames();
// Count total frames
console.log('Total frames:', allFrames.length);
// Loop through all frames
for (let frame of allFrames) {
  console.log('Frame Name:', frame.name());
  console.log('Frame URL:', frame.url());
}
await browser.close();
})();

```

12
34

Alternative: Count Frames with Locator

{ } Using Locators to Count Frames

```

import { Locator } from '@playwright/test';
// Count frames using locators
let allFrames: Locator[] = await page.locator('//frame').all();
console.log('Total frames:', allFrames.length);
// Get attributes of each frame
for (let frame of allFrames) {
  let src = await frame.getAttribute('src');
  let name = await frame.getAttribute('name');
  console.log(src, ':', name);
}

```



Nested Frames (Frame inside Frame)

{ } Handling Nested iFrames

```

import { test } from '@playwright/test';
test('nested frames', async ({ page }) => {
  await page.goto('https://example.com');
  // First level frame

```

```

const parentFrame = page.frameLocator('#parent-frame');
// Second level frame (nested inside parent)
const childFrame = parentFrame.frameLocator('#child-frame');
// Now interact with element in nested frame
await childFrame.locator('#button').click();
});

```

✓ Complete Example with Test

{ } Real-World Frame Test

```

import { test, expect } from '@playwright/test';
test('form inside iframe', async ({ page }) => {
  // Navigate to page
  await page.goto('https://www.formsite.com/templates/');
  // Click to open form with iframe
  await page
    .getByTitle('Vehicle-Registration-Forms-and-Examples')
    .click();
  // Get iframe locator
  const formFrame = page.frameLocator('iframe[id*="frame-one"]');
  // Fill form inside iframe
  await formFrame
    .locator('#RESULT_TextField-1')
    .fill('Naveen Automation Labs');
  await formFrame
    .locator('#RESULT_TextField-2')
    .fill('test@test.com');
  // Verify value inside frame
  await expect(formFrame.locator('#RESULT_TextField-1'))
    .toHaveValue('Naveen Automation Labs');
  // Access element outside iframe (main page)
  const header = await page
    .locator('h3.details__form-preview-title')
    .innerText();
  console.log('Header:', header);
});

```



PRO TIPS:

1. Always use `frameLocator()`:

→ It's the safest and most reliable method

2. No need to switch back:

→ Use `page.locator()` for main page elements

→ Use `frameLocator().locator()` for frame elements

3. Nested frames:

→ Chain `frameLocator()` for nested iframes

4. Wait for frame load:

→ `frameLocator()` auto-waits, but `frame()` doesn't!

✗ COMMON MISTAKES:

1. Using regular locators for frame elements:

✗ Bad: `page.locator('#inside-frame')`

✓ Good:

```
page.frameLocator('iframe').locator('#inside-frame')
```

2. Using `frame()` without null check:

→ Always check if frame is null when using `frame()`

3. Forgetting `frameLocator()` for nested frames:

→ Must chain `frameLocator()` for each level

4. Not waiting for frame to load:

→ Use `frameLocator()` which auto-waits



PRACTICE EXERCISE:

Task: Practice frame handling

Go to: <https://the-internet.herokuapp.com/iframe>

- 1. Use `frameLocator()` to access the iframe*
- 2. Clear existing text in the editor*
- 3. Type new text: "Learning Playwright!"*
- 4. Verify the text is updated*
- 5. Count total frames on the page using `frames()`*
- 6. Print frame name and URL*

Bonus: Create a test that handles nested iframes on a different website!



KEY TAKEAWAYS:

- ✓ *Frames are separate HTML documents within a page*
- ✓ *Use `frameLocator()` - it's the best method!*
- ✓ *`frameLocator()` auto-waits and never returns null*
- ✓ *`frame()` is not recommended - doesn't wait, can return null*
- ✓ *Use `frames()` to get all frames and count them*
- ✓ *Chain `frameLocator()` for nested iframes*
- ✓ *Access main page with `page.locator()`*
- ✓ *Access frame elements with `frameLocator().locator()`*

Happy Testing! 🚀

